# Architecture and protocols

Proposed Flutter mobile client over the existing Aegis SRE backend.

Mohamed Salem - 25058911 | Individual submission | MTM4681

Ahmed Harib | External contributor (outside the university) | Backend author

## System boundary

The repository contains the imported Python/FastAPI Aegis SRE implementation. The Flutter mobile client is a proposed presentation layer. Backend diagnosis, model calls, isolated verification and GitHub delivery remain on the server. The device presents state and evidence; it does not receive infrastructure credentials or deploy production changes.

## Selected pattern: MVVM

| Layer | Responsibility / examples |
|---|---|
| View | Flutter screens: SignIn, MyWork, InboxIncident, QueueReview, AgentList and AgentChat. Render state and forward user actions. |
| ViewModel | Own loading, content, empty, stale and error states; selected incident; tab state; action availability. Keep asynchronous workflow logic out of widgets. |
| Domain model | Service, Incident, VerificationResult, DeliveryResult, Conversation and Message. Conversation/message types are proposed additions. |
| Repositories | ServiceRepository and IncidentRepository coordinate remote data and bounded cache. Proposed AuthRepository and ConversationRepository isolate new capabilities. |
| Services / storage | HTTP JSON client, foreground SSE client, external PR launcher and timestamped local cache. Platform-secure session storage is proposed. |

## Data flow

Flutter View ↔ ViewModel ↔ Repository ↔ API / event services ↔ FastAPI
Repositories also read/write a bounded local cache. ViewModels expose typed state to views. This separation makes network mapping and failure handling testable without rendering UI, and lets the backend evolve independently of screen structure.

## Authentication and trust

GitHub sign-in is a new integration requirement. A browser-based OAuth flow should exchange authorization through the backend and issue an app session; service tokens remain server-side. Remote deployment needs HTTPS, authenticated sessions, per-user service access and validated redirects. These controls are not claimed as existing behavior of the inspected development listener.

# Backend orchestration

Existing subsystems and how the mobile client interprets their outcomes.

Mohamed Salem - 25058911 | Individual submission | MTM4681

Ahmed Harib | External contributor (outside the university) | Backend author

| Subsystem | Existing purpose |
| --- | --- |
| Telemetry / webhooks | Read database activity and accept/classify alerts. The FastAPI listener exposes status, service configuration, incident results, logs and events. |
| Database workflow | Inspect query/schema/plan context, generate candidate index SQL, apply SQL guard checks, and verify performance in an isolated Modal environment. |
| API workflow | Use specification differences or runtime error context to locate affected code, propose a patch and run configured target tests in isolation. |
| Model integration | Generate candidate changes and reports. Resource-intensive reasoning and provider credentials stay on the backend. |
| Delivery | Prepare changes and GitHub PRs after verification. The delivery layer can also report local simulation when credentials are absent. |

## Sequence and mobile state mapping

An alert or telemetry signal identifies a candidate incident. The backend checks for an existing matching PR, gathers context, generates a change and runs isolated verification. Failed verification can feed a bounded retry loop. Successful verification leads to a PR/report or a simulated local outcome. The mobile display should map this to explicit stages and retain the incident identifier throughout.

The current incident history is in-process in the inspected listener. Durable storage, reliable per-incident event correlation and typed progress messages are proposed extensions. The manual remediation route waits for the result; it is not an existing asynchronous job-ID interface.

## Evidence and delivery semantics

The UI distinguishes candidate verified, PR ready, verification failed, existing PR and simulated result. Backend labels such as REMEDIATED must not become a claim that production is fixed. The example 240 ms → 3.69 ms benchmark and 98.46% reduction are recorded upstream results, not a benchmark executed for this milestone. API results should show the actual targeted test count rather than imply complete application coverage.

## Out of scope

Automatic deployment, in-app merging, background push delivery, full administration and the future-agent catalog are outside the implemented milestone. Agent conversations and the inbox presentation are proposed client capabilities; live chat requires an additional backend contract.

# Interfaces and protocols

Existing endpoints, proposed transport requirements and integration purpose.

Mohamed Salem - 25058911 | Individual submission | MTM4681

Ahmed Harib | External contributor (outside the university) | Backend author

| Interface | Purpose / status |
|---|---|
| GET /api/status<br>GET /api/services | Existing JSON routes: current monitor/service status and configured services. Feed My Work and service context. |
| GET /api/incidents | Existing incident-result/history route. Map to Inbox and Queue with typed client models. |
| GET /api/stream/events | Existing Server-Sent Events (SSE) stream for one-way foreground updates. Proposed client handles reconnect and event correlation. |
| GET /api/logs?since_id=… | Existing incremental log retrieval. Supports activity display and refresh after reconnect. |
| POST /api/remediate | Existing explicit remediation entry point; waits for worker result. Proposed client prevents duplicate submissions and confirms action state. |
| POST /webhooks/alert | Existing alert ingestion endpoint used by monitoring systems; not a mobile chat endpoint. |

| Connection | Protocol / rationale |
|---|---|
| Mobile ↔ backend | JSON REST and SSE over HTTPS in the proposed remote deployment. REST reads/actions; SSE receives foreground progress. |
| Backend ↔ PostgreSQL | PostgreSQL driver and read-only inspection/telemetry access; candidate execution belongs in the isolated test environment. |
| Backend ↔ model / Modal | Provider HTTP API for generation; Modal SDK/cloud interface for isolated execution and verification. |
| Backend ↔ GitHub | Authenticated GitHub API for branches, commits and PRs. Mobile opens confirmed PRs through HTTPS. |

## Failure handling and validation plan

Use explicit timeouts and errors; reconnect streams with bounded backoff; refresh authoritative incident state before retrying an action. Mark cached data stale. Test payload mapping, simulation/live gating, duplicate-action prevention and verification-failure presentation. Chat must not execute unconfirmed infrastructure changes.